Forecasting county-level power outage ratios from archived ECMWF forecasts

Huawei Tech Arena 2026 — Topic 2, AIDC Power-Supply Risk Forecasting, Phase 1

Author: Tommaso Facchin (solo entry) **Tasks:** A (48 h / 1 h) and B (6 h / 15 min) **Test window:** 1 Sep – 30 Nov 2025
Date: 24 August 2026

1. What we built, and what came out of it

The system predicts `x = customers_out / total_customers` for five US counties, at every lead time both tasks require, from archived ECMWF IFS HRES forecast runs plus the observed EAGLE-I outage history available at each issue time. The submitted `x` is a fraction of `MCC.csv` as published, the denominator the organizers confirmed on 24 August that they grade against; the model trains on a repaired one, and Section 3.4 is why both are necessary. It is a single LightGBM regressor that predicts the residual from persistence rather than the level, one model covering both tasks, with lead time as an explicit feature, blended toward a plain persistence forecast by a weight fitted per lead bucket. The submitted file contains 66,120 rows: 93 daily Task A batches of hourly means and 365 six-hourly Task B batches of 15-minute instants, five counties each, generated from 93 distinct archived forecast runs. It is built from 388 forecast runs across 102 counties — 2.80 million rows — spanning September 2024 to August 2025, of which the 2.30 million before 1 May 2025 are used for training and the rest held out.

The result that shaped this submission is what the model is asked to predict. Predicting `x` directly under a Tweedie objective — the textbook choice for a non-negative target with an atom at zero — produced a model with almost no skill: on the seasonally matched autumn window it beat always-zero by 1.1 %, its error barely varied with lead time, and its largest prediction anywhere was 0.18 against a truth reaching 1.0. Predicting instead the **residual from persistence** under squared loss takes the same features and hyperparameters to **+30.1 %** and restores the full dynamic range. The diagnosis, in Section 5, is that a log link on a target 69.9 % exact zeros collapses toward a small constant.

Two consequences run through the rest of this report. The blend with persistence, which previously supplied essentially all of the system's skill, is now worth 1.5 points on top of a model that works: its fitted persistence weights fall from 1.00 / 1.00 / 0.95 / 0.90 / 0.55 to 0.40 / 0.55 / 0.50 / 0.40 / 0.50, and the shortest bucket — the whole of Task B, two-thirds of the submitted rows — goes from pure carry-forward to 60 % model. And the model's attention moves onto the weather: county identity falls from 19 % of split gain to **8.9 %** while all weather rises from 28 % to **42 %**.

2. Choosing the five counties

The guidelines let each team pick its own five counties and ask for the choice to be justified. Our first instinct — rank all ~3,050 EAGLE-I counties by how often they report an outage and take the top five — is a trap, and it is worth describing because the trap is invisible in the output. Ranking by the fraction of 15-minute intervals with any nonzero `customers_out` returns Harris County TX, Miami-Dade, and their peers at rates of 99.9 %. Those counties are not unusually fragile. They are unusually large, and in a county with three million customers there is essentially always somebody off supply somewhere. What that ranking recovers is population, not weather risk.

So we ranked instead on the fraction of intervals with `x` above 1 %, which requires an event large enough to be visible at county scale. On top of that we required continuous EAGLE-I coverage across the window (a reporting gap and a quiet county look identical in a sparse event log), a denominator above 5,000 customers so that `x` is not dominated by single-household noise, and diversity of weather regime across the final five rather than five variations of the same climate.

The activity statistics come from January–August 2025 only, not the multi-year climatology we wanted: the forecast archive that supplies our features starts in March 2024 and the API budget did not stretch back further. That window at least stops before the evaluated one opens, so the selection never looks at a month it will be scored on.

Table 1 — The five reporting counties. Activity statistics cover 1 Jan – 31 Aug 2025 at 15-minute resolution, 23,328 intervals per county. "Significant" means `x > 0.01`. "Customers" is the reconciled denominator of Section 3.4, which the model trains on; the submitted numbers are converted to MCC before they leave `predict.py`.

FIPS	County	State	Customers	Significant	Max x	Regime, and why it is in the set
72013	Arecibo	Puerto Rico	191,803	15.2 %	0.725	Tropical/Caribbean. A chronically fragile grid, and still the strongest signal in the set once the denominator is corrected.
37119	Mecklenburg	North Carolina	588,615	0.4 %	0.023	Piedmont/south-east. Hurricane remnants inland plus severe summer convection; Charlotte metro, so a large and well-reported denominator.

FIPS	County	State	Customers	Significant	Max x	Regime, and why it is in the set
54005	Boone	West Virginia	13,090	12.4 %	0.343	Appalachian interior. Ice storms and thunderstorm downbursts on wooded terrain; no maritime influence at all.
26097	Mackinac	Michigan	8,757	12.2 %	0.866	Great Lakes. Winter storms and repeated near-total local outages — the highest-variance county of the five.
22071	Orleans	Louisiana	212,936	0.9 %	0.245	Gulf Coast. Included deliberately despite a thin January–August signal: peak hurricane season falls inside the test window, and leaving the regime out because our proxy window misses it would be exactly the wrong inference.

Orleans is the one selection that a pure ranking would not have made, and we would defend it as the most important of the five. September to November is the Gulf hurricane season. A selection procedure calibrated on January to August will systematically under-rate Gulf counties, because their season has not started yet in the data it looks at. Correcting for that by hand is not cherry-picking; refusing to correct for it would be.

> One caveat we would rather state than bury. The ranking ran before the denominator work of Section 3.4 and used `MCC.csv` throughout, so two of the five were ranked on a denominator we later found to be physically wrong: Mecklenburg's too small by 20.9×, which carried it in as the strongest mainland signal, and Arecibo's by 4.7×, which is why its `x` saturated at 1.000. On the repaired denominators of Table 1, Arecibo survives as the strongest county in the set (43.5 % → 15.2 % significant) and Mecklenburg becomes the quietest (13.8 % → 0.4 %). We kept it: the regime argument that shortlisted it is untouched by the arithmetic, but a county that entered on a number we later corrected is not there on the stated criterion. Read Mecklenburg as a selection we disclose, not one we defend — with one thing added since. Those ranking numbers are MCC ratios, and MCC is the unit the submission is scored in, so Mecklenburg's scored `x` is genuinely active while its physical `x` is quiet. Both are facts about it.

3. Data acquisition and preprocessing

3.1 Ground truth: finding EAGLE-I, and then reading it correctly

The competition materials point at the ORNL/OSTI record, which routes to a Globus endpoint covering 2014–2022 and requiring an account. That is not the only scriptable route, and believing it was cost most of a day: the complete 2014–2025 release, 17 files and about 11.6 GB, sits on figshare as article 24237376 (version 4, February 2026), and every annual file is an anonymous HTTPS GET against `ndownloader.figshare.com`. Our downloader resolves that file table from the figshare API rather than hard-coding it, and fetches whichever years are asked for.

The annual files do not share a schema, and the drift is silent rather than fatal. Files from 2014–2022 and 2025 use `fips_code, county, state, customers_out, run_start_time`. The 2023 file calls the fourth column `sum`. The 2024 file adds a sixth column, `total_customers`. A straightforward `pd.concat` across years therefore produces a `customers_out` column that is entirely null for 2023 without raising anything at all. Our loader normalizes headers through an explicit alias table before concatenating, and raises on an unrecognized rename instead of degrading into nulls.

3.2 From sparse events to a dense target

EAGLE-I emits a row only when `customers_out > 0`. Everything else is implicitly zero, and a model trained only on rows that exist would never see the negative class it mostly has to predict. Preprocessing reindexes each county onto a complete 15-minute grid across the period of interest and zero-fills the gaps, then computes `x` and clips it to [0, 1]. The clip now only catches the 111 counties of Section 3.4 whose numerator no candidate denominator can hold; everywhere else the denominator was repaired instead.

About 6.5 % of raw EAGLE-I rows carry an explicit null `customers_out`. That is not the same as an absent row. An absent row means no outage; an explicit null means the upstream ETL could not get a reading. Coercing nulls to zero would encode “we do not know” as “confirmed no outage”, and would bias the model toward under-predicting risk during exactly the periods when utility reporting is itself degraded — which is plausibly correlated with the severe weather we are trying to predict. We treat the two cases differently: a null in the *target* means the row is dropped, since there is nothing to train or score against; a null in an *autoregressive input* is left as a null and handed to LightGBM, which learns a default split direction for missing values. Imputing a fabricated number there would be strictly worse.

3.3 Weather: the Single Runs archive and its real rate limits

The guidelines require that prediction-time weather be the forecast genuinely available at the issue time, which rules out the Historical Forecast API (it stitches together only the first hours of successive runs and so does not preserve lead-time-specific values) and rules out observed reanalysis. We use Open-Meteo's Single Runs API with `models=ecmwf_ifs`,

which returns a specific archived initialization of IFS HRES at 9 km. That archive begins on 14 March 2024; every other model in the endpoint is archived only from April 2026 and cannot overlap the test window. So IFS HRES is not a preference here, it is the only option, and its start date is the hard boundary on how far back training data can go.

The binding constraint on this project turned out to be the endpoint's rate limits. Against published free-tier terms of 10,000 calls a day, the Single Runs endpoint enforces three much tighter undocumented windows: per minute, per hour (~62 minutes to clear), and a daily cap that in practice cut in around 110 to 130 calls. The same quota pays for generating the submission, so it shaped the design. Two responses followed, both of which cost accuracy:

- The training sample was cut to 102 counties, stratified by Census region and by activity tercile with a cap of three counties per state per cell (Texas alone has 151 eligible counties, and an unstratified draw would have produced a Texas weather model). Runs were taken at 00Z and 12Z rather than all four daily cycles: every day across September–December 2024, and one day in two through 2025.
- Task B shares Task A's forecast run rather than fetching a fresher one every six hours. Fetching per-issue-time would have cost 367 distinct calls for the test window; sharing one run per calendar day costs 93. The forecast a Task B batch consumes is therefore 12 to 30 hours old rather than 6, which is still a forecast genuinely available at issue time and so still compliant, but it is a real skill concession made for a quota reason. Reverting it is a one-line change if the quota stops being scarce.

3.4 The denominator: two of them, and which one the submission is written in

`x` is a ratio, so the customer count in its denominator multiplies every number we predict. The guidelines call that value "fixed and maintained by the organizers", and for most of this project we read it as a number we would never see, which made the denominator the one uncontrolled risk in the submission. It is `MCC.csv`. Asked whether scoring divides by `total_customers` exactly as `MCC.csv` gives it even where the recorded `customers_out` exceeds it, they answered on 24 August: *"even the official dataset could have mistake, when grading we will ignore such timestamp"*, alongside an earlier *"MCC.CSV in the first link is the total customer number listed in the fips code"*. Only a grader dividing by MCC as published produces a timestamp worth ignoring, so that settles the unit: **the submitted number is a fraction of MCC**, and the intervals MCC makes impossible are dropped from scoring rather than repaired.

We still cannot train on it. Summed per state against `coverage_history.csv`, MCC reproduces the published state total to within a few percent almost everywhere — but North Carolina lands at 1.82 M against 5.30 M, and Mecklenburg, one of our five, carries 28,172 customers against 588,615 in EAGLE-I's own 2024 annual file: a factor of 20.9 on every `x` we computed for it. Switching wholesale to the 2024 column is wrong in the other direction, giving Arecibo 191,803 against MCC's 41,122 in a municipio of ~87,000 residents. So the reconciled table takes each state's allocation from whichever candidate reproduces its published total more closely (Kansas, North Carolina and Texas move), then overrides that per county wherever the largest `customers_out` on record exceeds the chosen denominator — a ratio on [0, 1] cannot have a numerator 3.4 times its divisor. That repairs **149 counties**, Arecibo among them; 111 more, where neither candidate holds the numerator, take the larger and are flagged `unresolved` (`data/processed/denominator_reconciliation.md`).

So the project carries two denominators. The reconciled one is what the target grid, the training table, the model and the blend all divide by: a target that saturates at `x = 1` for the whole duration of a storm teaches the model that every large event is the same size. MCC as published is what the submission is read in. Since `x` is linear in the reciprocal of its denominator, converting between them is one constant per county applied in one place (`to_grading_units()` in `predict.py`): three of our five are multiplied by 1, Mecklenburg by 20.89 and Arecibo by 4.66.

Until the answer came we had left the submission in reconciled units, on the argument that under-predicting `x` is the cheaper error on a target made mostly of zeros. That was wrong, measurably: on the equivalent autumn window of 2024 the grader discards only 1.7 % of Mecklenburg's intervals and 0.2 % of Arecibo's for exceeding 1, and across the rest the truth in MCC units averages 0.0151 and 0.0300 against the 0.00072 and 0.00643 we would have submitted — a systematic 21× and 4.7× under-prediction on two of the five counties.

3.5 Timestamp alignment and spatial matching

One alignment question is *what a Task A row is*, not when it is. EAGLE-I records every 15 minutes and Task A is scored hourly, so the organizers asked for hourly mean aggregation: a row here is the mean over the four quarter-hours its label opens, 03:00 meaning [03:00, 04:00). It matters — across event hours in our five counties over January–August 2025 the hourly mean and the value at :00 differ by a fifth of the quantity predicted, an RMSE 18 % of this system's autumn figure — and needs no second model, since the mean of unbiased estimates of the four instants estimates their mean without bias. Task B rows are 15-minute instants already.

Everything internal is naive UTC. EAGLE-I's `run_start_time` is parsed as UTC and never localized; the Open-Meteo response comes back timezone-aware and is converted to naive UTC at the point where the weather table is renamed for joining, because a tz-aware and a tz-naive column merge into an empty result rather than an error. Output timestamps are formatted as ISO 8601 with an explicit `Z`. FIPS codes are strings everywhere, zero-padded to five characters at every load, which matters for Puerto Rico and for every state whose code starts with a zero.

Spatial matching is the simplest thing that is defensible: each county is represented by its internal-point centroid from the 2025 US Census Gazetteer, and the forecast is requested at that single coordinate. Phase 1 targets counties rather than sites, so there is no site-to-county assignment to make; the approximation is that one point represents a county's weather, which for a lake-shore county like Mackinac is a real simplification. Area-averaging over a grid of points per county would be better and costs a multiple of the API budget we did not have.

4. Features

Fifty features in five groups. The design principle throughout is that a feature must be a function only of data timestamped at or before the issue time, checked by a single shared assertion (`code/features/causality.py`) rather than by scattered ad-hoc guards.

Raw forecast fields (12): 10 m wind speed, gusts and direction; precipitation, rain and snowfall; 2 m temperature and dew point; surface pressure; CAPE; cloud cover; freezing-level height.

Derived weather (11). Gust cubed, because wind loading on a conductor or a tree scales roughly with the cube of speed and the linear gust value compresses exactly the range that breaks things. Centered rolling gust maxima over ±3 h, ±6 h and ±12 h, which answer "is this hour inside a windstorm" rather than "is this hour windy". Cumulative precipitation over the preceding 6 and 24 hours, as a proxy for soil saturation, since saturated ground plus gusts is what uproots trees. A wind×rain interaction term. Three-hour pressure change and one-hour gust jump for rate-of-change. All of these are computed strictly inside a single forecast trajectory, which is why a centered window is causally fine: those hours were disseminated together in one run, and causality here is run time against issue time, not neighboring hours within a forecast that has already arrived.

Climatological exceedance (12). Per-county p95, p99 and p99.9 of gusts and precipitation, plus boolean flags for exceeding each. These are what let one model serve both a Gulf county and an Appalachian one: 60 km/h means something different in each, and the exceedance flag says how unusual the value is locally instead of how large it is absolutely. The thresholds are fitted once on the training portion of the split only, saved with the model, and reapplied unchanged at prediction time. Recomputing them at serving time is a subtle and destructive bug: a single 62-hour forecast batch has a p95 gust roughly half the training p95 (26.4 against 49.7 km/h for Orleans), so the exceedance features would quietly mean something different in production than in training.

Autoregressive outage state (11). Observed `x` at issue time and at −15 min, −30 min, −1 h, −2 h and −6 h; one- and two-hour trends; the 24-hour rolling maximum; a flag for an outage in progress and the length of the current run. The guidelines restrict only the *weather* input to what a forecast could have provided; observed outage history up to issue time is realistic, since EAGLE-I and the ODIN feed are near-real-time, and it is the main source of genuine sub-hourly skill for Task B. IFS has no native 15-minute output, so the weather a Task B row sees is interpolated from hourly values and carries no true sub-hourly information. Whatever timing skill Task B has comes from this group.

Temporal and static (6). Hour of day and day of year as sine/cosine pairs, lead time in hours, county FIPS as a categorical, and the customer count.

5. Model and training

One LightGBM regressor with `objective=regression`, 500 trees, learning rate 0.05, 63 leaves, minimum 50 samples per leaf, 0.8 column subsampling (a row-subsampling fraction is configured but inert, since LightGBM bags only when `bagging_freq` is set; every tree is grown on all rows). It does not predict `x`. It predicts the **residual from persistence**, `x(target) − x(issue)`, and the submitted level is that residual added back to the observed state at issue time. Both tasks are served by the same model with lead time as a feature, since a per-task split would halve the training data for no structural reason.

This replaced a Tweedie head on the level, and it is the largest single improvement in this submission. That head had almost no skill: on the autumn holdout it beat always-zero by 3.8 % in the 0–6 h bucket, its RMSE barely moved across lead time (0.032824 at 6 h, 0.033754 at 72 h — a model no longer conditioning on the present), and its largest prediction anywhere was 0.18 against a truth reaching 1.0. The cause is the objective, not the features: Tweedie carries a log link, and on a target 69.9 % exact zeros the fitted response collapses toward a small constant, so `x_at_issue` being the highest-gain feature did not mean the model could reproduce it. The residual target removes that by construction — centred near zero, squared loss appropriate, persistence recovered exactly when the model outputs zero. Measured on the same holdout in the grading denominator over the five reporting counties, after the organizers' rule dropping timestamps whose truth exceeds 1, pooled RMSE falls from 0.068473 to **0.049901, a 27.1 % reduction**. Extra capacity does not help — 1,500 trees at 127 leaves and 3,000 at 255 are both worse than 500/63 — as expected if the objective was the binding constraint.

The split is temporal and never random: training on rows before 1 May 2025, validating on everything after. A random split would put two rows from the same storm fifteen minutes apart on opposite sides and make every validation number meaningless. The held-out period is not an unrepresentatively calm stretch: 36.0 % of its rows are positive

against 28.8 % in training. Because the weather download kept running to the end, every run it added after 1 May lands in the holdout rather than in training — the evaluation set is 502,777 rows against 2.30 million trained on.

Two decisions came out of bugs rather than design. There is no early stopping: an earlier version registered no validation set through a keyword-name error, stopped after 25 of 500 trees and deployed a stump with a maximum prediction of 0.012 against today's 0.297. We removed early stopping rather than repair the wiring, because stopping on the validation set and then reporting metrics on it would make every number here optimistic. And the model ships as a bundle carrying the fitted state its features depend on — FIPS category ordering, climatology thresholds, and now which quantity the booster predicts — because re-deriving any of them at prediction time silently answers a different question.

5.1 Blending with persistence, and the season it is fitted in

The lead-time curve showed persistence beating the model below about twelve hours and losing badly beyond it, so a single predictor is the wrong shape for a submission whose two tasks sit on opposite sides of that crossover. There is also a specific train/serve mismatch behind it. During training, issue time and run time are the same instant, so the model's `lead_hours` feature means both "how far ahead is the target" and "how stale are the autoregressive features". At serving time the two come apart: a Task B row asks for a fifteen-minute-ahead prediction while carrying `lead_hours` between 12 and 36, because that is the age of the forecast. The model learned to discount autoregressive features at long lead, and nothing in its input says that this time they are minutes old. Persistence has no such confusion.

So we fit a weight per operational-lead bucket that minimizes RMSE between the model and a plain persistence forecast, and apply it at prediction time on the operational lead. RMSE rather than MAE, because on a target that is 69.9 % exact zeros, MAE is minimized by collapsing toward zero and a blend tuned on it would simply pick whichever component predicts less.

Which held-out period the weights are fitted on still matters, though less than it did when the blend was carrying the whole system. Fitted on the forward-in-time split, which lands in May to August, the weights come out at 0.75 / 0.65 / 0.75 / 0.80 / 0.70 across the five lead buckets. Fitted by the identical procedure on the autumn holdout described in Section 7.1, they come out at **0.40 / 0.55 / 0.50 / 0.40 / 0.50** — that is, in the season being scored the model is the stronger component nearly everywhere, and persistence is worth about half the prediction rather than all of it.

Under the Tweedie head these weights were 1.00 / 1.00 / 0.95 / 0.90 / 0.55 — persistence took the two shortest buckets *outright*, so the whole of Task B was a pure carry-forward. On the autumn window the spring-fitted weights reach 0.026524 and the autumn-fitted 0.023188 (**+31.6 %** against always-zero); out of season the autumn weights now cost nothing. They are fitted against a model retrained on 2025 alone, never against the deployed model, whose autumn predictions are in-sample and would have understated what this measures.

The carry-forward decay now earns its place, where before it was chosen and almost worthless. Rather than assert that a county with 4 % of customers out now still has 4 % out in two days, the fit searches an exponential restoration decay jointly with the weight. It picks one in the two longest buckets, at half-lives of 96 h and 48 h, worth **1.07 %** overall against 0.06 % under the previous head; restricted to flat carry-forward it would drop persistence to 0.20 and 0.00 there rather than decay it.

One caveat we would rather state than bury: autumn 2024 contains Helene and Milton, so these weights would be too persistence-heavy for an unusually quiet autumn. The downside is bounded, since during calm periods `x_at_issue` is near zero and a heavy persistence weight shrinks predictions toward zero, the direction this target rewards anyway.

6. Handling the class imbalance

In the training table 69.9 % of rows have `x` exactly zero, 30.1 % are nonzero, and only 5.7 % exceed 0.001. The mean is 0.00180. Any error metric averaged over all rows is therefore a statement about the zeros, and an always-zero forecast is a genuinely strong competitor on MAE rather than a strawman. We address this in four places.

The target formulation removes the zero mass rather than modelling it. Because the model predicts `x(target) − x(issue)`, most zero rows are ones where the county was *already* at zero and stays there, entering the objective as a residual of zero rather than as a point mass the response must reproduce. The previous Tweedie head instead absorbed the zero mass inside a compound Poisson-gamma objective — the textbook fit for a target with an atom at zero — and in practice collapsed toward a small constant (Section 5).

Evaluation is stratified rather than pooled. Every result in Section 7 is reported both over all rows and restricted to rows where the observed `x` exceeds 0.001, and the always-zero baseline appears in every table so that a reader can see how much of a number is skill and how much is arithmetic on zeros. We also report maximum prediction per model, because a model that looks competitive on average while never predicting above 1 % of customers out is useless for the actual application.

The hurdle formulation in Section 7 attacks the imbalance structurally by splitting the question in two: a classifier for whether an event happens at all, and a severity regressor trained only on positives, so that the severity head never sees a zero and is not dragged toward one.

7. Results

Unless stated otherwise, numbers below are on the forward-in-time held-out period after 1 May 2025 (502,777 rows, of which 37,367 have observed x above 0.001), using a model that never saw a row from that period, with thresholds and category encodings fitted only on the earlier portion. Section 7.1 then repeats the comparison on a seasonally matched autumn window, which is the one we would trust for the test period. Every error figure here is in the reconciled denominator of Section 3.4, the unit the model is fitted in; the submitted file is converted to MCC per county before it is written.

Table 2 — Baselines, model, and feature-group ablations. Each ablation is a full retrain with that group removed, so its effect is marginal given everything else, not a univariate correlation. Lower is better in the first three columns.

Predictor	MAE	RMSE	MAE, events	Max pred.	ΔRMSE vs full
always zero	**0.001078**	0.010663	0.013801	0.000	−0.68 %
persistence (x at issue time)	0.001554	0.011963	0.013634	0.309	+11.44 %
per-county climatology	0.002620	0.011759	0.013320	0.042	+9.54 %
full model (delta)	0.002102	0.010735	**0.012799**	0.297	—
− raw forecast fields	0.001921	0.010681	0.012645	0.316	−0.50 %
− derived weather	0.002161	0.011028	0.012856	0.311	+2.73 %
− climatological exceedance	0.002091	0.010939	0.012721	0.306	+1.90 %
− autoregressive state *and persistence base*	0.002027	0.011031	0.013236	0.134	+2.75 %
− temporal encodings	0.002404	0.012247	0.013253	0.307	+14.08 %
− static county features	0.001941	**0.010463**	0.012677	0.276	−2.54 %
hurdle, `τ = 0`	0.001069	0.010550	0.013226	0.064	−1.72 %
hurdle, `τ = 0.001`	0.001147	0.010522	0.012933	0.091	−1.99 %

Read this table knowing which window it is: May–August, the calm season, where always-zero is very hard to beat and even plain persistence scores *worse* than predicting nothing (0.011963 against 0.010663). The full model wins the one column that is about outages rather than zeros — event-restricted MAE, 0.012799, the best in the table — and gives up 0.68 % of pooled RMSE to always-zero, because predicting a small positive number everywhere costs MAE on 465,410 zero rows to buy accuracy on 37,367 event rows. Section 7.1 repeats this on the autumn window, where the ordering is not close.

The ablations are no longer marginal, itself a consequence of the reformulation: a model with genuine skill has something to lose when a group is removed. Temporal encodings dominate at +14.08 %; the autoregressive block — removed here *together with* the persistence base, since `x_at_issue` enters the deployed system both as a feature and as the level the residual is added to — costs +2.75 %, alongside derived weather at +2.73 % and climatological exceedance at +1.90 %. That autoregressive figure looks small only because of the window: in a calm season the state at issue time is usually zero, the same fact that puts persistence below always-zero two rows up. Raw forecast fields (−0.50 %) and static county features (−2.54 %) come out negative; we keep both, since correlated groups let neighbours absorb what is dropped, which makes the split-gain share the better guide.

Table 3 — The seven highest-gain features, as a share of the model's total split gain.

Feature	Gain share	Group
`x_at_issue`	11.4 %	autoregressive
`doy_sin`	10.4 %	temporal
`fips_code`	8.9 %	static
`gust_clim_p999`	7.7 %	climatology
`gust_clim_p95`	6.4 %	climatology
`x_lag_1h`	5.5 %	autoregressive
`doy_cos`	4.0 %	temporal

This table is the clearest single piece of evidence for what the old objective was doing. County identity was 43 % of split gain before the autumn runs were densified and 19 % after; it is now **8.9 %**, while all weather together rises from 28 % to **42.3 %**. A model forced to reproduce the *level* spent most of its capacity learning which county a row belongs to, because county is what sets the level. Handed the level for free and asked only for the change, the same features under

the same hyperparameters redistribute that capacity onto weather — the quantity the task is actually about. The weather features carrying the weight are the climatological exceedance thresholds (`gust_clim_p999`, `gust_clim_p95`): gusts read relative to what is normal for that county rather than in absolute terms.

7.1 Skill against lead time

The forward-in-time split above lands in May to August. The window being scored is September to November, and the two are not interchangeable: autumn RMSE on this target is more than twice the late-spring figure, because autumn outages are larger. So we re-ran the same comparison on a seasonally matched window — train on 2025, evaluate on September–November 2024, which the deployed model has seen but this probe's model has not. Time runs backwards in that arrangement, which is fine for a seasonal-transfer question and is used for no model selection: it measures how a model fitted outside autumn behaves inside it.

Table 4 — The same predictors in two seasons. Skill is RMSE reduction against always-zero within that season. Persistence is scored where x at issue time is defined (all autumn rows; 501,160 of 502,777 reference rows).

Predictor	Autumn RMSE	skill	Reference RMSE	skill
always zero	0.033903	—	0.010663	—
persistence alone	0.026046	+23.2 %	0.011968	−12.2 %
model alone	0.023705	+30.1 %	0.010735	−0.7 %
blend, spring-fitted weights	0.026524	+21.8 %	0.010285	**+3.5 %**
blend, autumn-fitted weights (submitted)	**0.023188**	**+31.6 %**	0.010555	+1.0 %

The model alone now carries +30.1 % of the +31.6 % the submitted system reaches; under the previous head those figures were +1.1 % and +30.4 %, i.e. the blend supplied essentially all the skill. It is still worth its 1.5 points, but now improves a component that works rather than rescuing one that did not. Persistence still swings from +23.2 % to −12.2 % between seasons, but the submitted configuration no longer follows it down, scoring +1.0 % out of season where the old one gave up 2.6 %. The spring-fitted weights are now the wrong choice in autumn (+21.8 %, below the model alone), leaning on persistence where the model has become the better component.

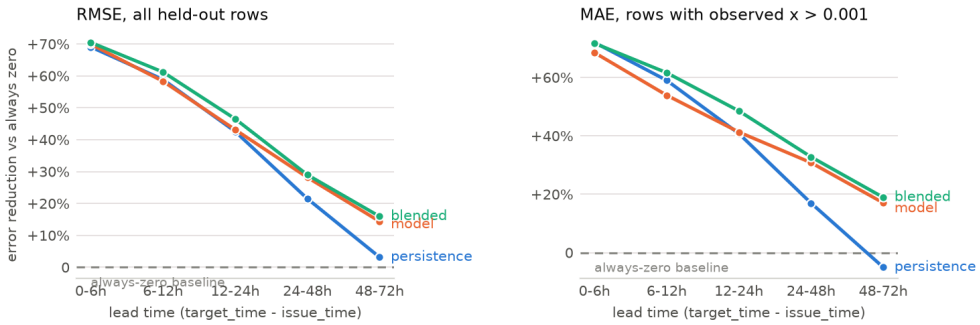

Figure 1. Error reduction against always-zero per lead-time bucket, on the 1,318,044 rows of the seasonally matched autumn window. Raw error per bucket is not comparable across buckets on this target — events are rare and clustered, so each bucket catches a different set of storms — which is why this is plotted as a skill score.

Figure 1 is the curve the guidelines say scoring will look at, in the season it will be scored in. Persistence decays from +69 % at 0–6 h to +3 % past two days, which is the behaviour a persistence forecast should have. The model now tracks it at short lead (+70 % in the first bucket) and separates from it as persistence decays, holding +14 % at 48–72 h where persistence has fallen to +3 %: it has learned the decay rather than inheriting it. Under the previous head this line was flat and small — +4 % at the shortest lead, fading to nothing at the longest — and the entire shape of this figure came from persistence. The blend stays above both at every bucket, not between them: +70 / +61 / +47 / +29 / +16 %. That is the shape we want for a submission whose Task B lives entirely in the first bucket and whose Task A runs out to the fourth.

7.2 The hurdle alternative

The hurdle model is the guidelines' class-imbalance question posed as an experiment: instead of one head on the level, a binary classifier for $P(x > \tau)$ times a severity head trained only on rows above τ. It was built to fix the Tweedie head's lack of dynamic range, and on the calm reference split it still leads on pooled metrics ($\tau = 0$ gives the best pooled MAE of anything tried, 0.001069). The autumn window decides against it, and now decides clearly: as standalone models the delta head scores 0.023705 against the hurdle's 0.032971 and 0.033566, a 28 % gap, where the Tweedie head it replaced was slightly *behind* both. Blended under weights refitted for each head they land at 0.023188, 0.023589 and 0.023592 — a spread of 1.74 %, against 0.05 % before, because persistence used to carry so much of the blend that the head could

not matter. It now can, and the single residual model wins on the season being scored while remaining one model to serve rather than two.

8. What limits these numbers

The training set is the limitation that matters, and it shrank at every stage of the week for reasons that were each individually defensible. A multi-year window became eighteen months once the IFS archive's March 2024 start was confirmed; the per-hour rate limit then cut the county sample from ~3,050 to 102 and the run cadence to every other day; the daily quota, found last, capped how many runs we could add. The last days of the project bought the cadence back for September–December 2024, the block that matters most. The table behind these numbers holds 388 forecast runs across 102 counties in 40 states, spanning 1 September 2024 to 4 August 2025; the model is trained on the 2.30 million rows of it that precede 1 May 2025.

The most obvious weakness in the earlier version of this submission was that its training window contained no Atlantic hurricane season while the evaluated window is mostly hurricane season. That is now closed: September to December 2024 is in at full 00/12Z cadence, bringing Helene and Milton with it, and weather takes 42 % of split gain against 28 % before. What the autumn data mainly bought, though, was not a better model but the ability to *measure* in the right season — and that is what produced both corrections this submission rests on: the blend weights, worth 12.6 % RMSE against weights fitted out of season, and the target reformulation, whose failure was invisible on the late-spring split where the model looked merely unremarkable rather than broken.

Which leaves an honest statement of what the submitted predictor is. The effective persistence weight, after the fitted restoration decay, is 0.40 in the shortest bucket and 0.31 at one to two days out — **0.40** averaged over the submitted rows, against 0.91 under the previous head, where the accompanying admission was that most of what we submitted was simply the observed state carried forward. That is no longer true. What has not changed is that the state at issue time remains the most valuable input the system has: it is now the base the model corrects rather than a component blended in afterwards, which is a better use of it, not a smaller one.

Beyond the blend-weight caveat Section 5.1 states and bounds, three smaller ones. The climatological quantiles are fitted on months rather than decades, so “p99 gust” here means “p99 of what this county saw in the training window”. Task B's weather is interpolated from hourly IFS output and contains no genuine sub-hourly information. And the denominator is now known rather than guessed, but knowing it does not make it right: MCC is too small for Mecklenburg and Arecibo, so their largest events push the ground truth above 1 and leave the scored set entirely. The two counties with the most severe outages in our five are graded almost wholly on their quiet hours, and the scored subset is easier than the record we report skill on.

9. Data sources and licenses

Every source used, with the license verified at the source on 21 August 2026 rather than assumed.

Table 5 — Data sources and licenses.

Source	Use	License
EAGLE-I Recorded Electricity Outages 2014–2025 (Oak Ridge National Laboratory). Brelsford, Tennille, Myers, Chinthavali, Tansakul, Denman et al., figshare article 24237376 v4, 25 Feb 2026. `doi:10.6084/m9.figshare.24237376.v4`	Ground-truth outage counts; `MCC.csv` and `coverage_history.csv` for the denominator.	**CC BY 4.0**, as returned by the figshare API for that article.
Open-Meteo Single Runs API `single-runs-api.open-meteo.com`	Delivery of archived ECMWF IFS HRES initializations, the sole weather input for both training and prediction.	**CC BY 4.0** under the free non-commercial tier, per Open-Meteo's Terms of Use. Attribution: *Weather data by Open-Meteo.com*.
ECMWF IFS HRES open forecast data (originating model behind the above)	The forecast fields themselves.	**CC BY 4.0** plus the ECMWF Terms of Use: “a subset of ECMWF real-time forecast data from the IFS and AIFS models is made available to the public free of charge … governed by the Creative Commons CC-BY-4.0 licence”, subject to attribution.
US Census Bureau 2025 County Gazetteer `2025_Gaz_counties_national.txt`	County internal-point centroids, the forecast coordinate for each county.	US Government work, not subject to domestic copyright (17 U.S.C. §105) — public domain.

Code, execution order and expected runtimes are in `README.md`. `predictions.csv` is regenerated end to end by `python code/predict.py`, which validates the file it writes.